# Graf: Project Graph for MCP

Product Requirements Document, v0.2 (merged with ChatGPT's Graf PRD v1.0) | Author: Gurjotpal | Status: Draft

## 1. Summary

AI agents working through MCP re-read the same files at the start of every session and often several times inside one session. This burns tokens, adds latency, and still leaves the agent with a shallow picture of the project.

Graf parses a project once into a compact **knowledge graph** (files, classes, functions, routes, models, and the relationships between them). It keeps that graph up to date automatically, versions it, lets the human edit it, and serves it to any MCP client through a small server. The agent reads a tiny always-available overview and pulls exact details on demand instead of re-reading source files.

**Working name:** Graf.

**One-line pitch:** a living, editable, versioned map of your codebase that agents can query instead of re-reading.

## 2. Problem

- Agents have no persistent memory of a codebase, so every session starts with exploratory reads (list folders, open files, grep).
- Reading whole files to answer small questions ('who calls this function?') wastes most of the tokens.
- Agents lose track of structure in long sessions and re-read to re-verify.
- Existing tools (repo maps, code search servers) are either read-only, English-summary-only, cloud-based, or heavy to set up. Few store human decisions ('why we chose X'), and almost none let the user correct the graph.

## 3. Goals and Non-goals

### Goals

1. Cut tokens spent on orientation and navigation by 80 to 95 percent on typical tasks (to be benchmarked, see section 12).

2. Zero-config: run one command in a project and it works.
3. Always fresh: graph updates within about a second of a file save.
4. Never lose history: every meaningful state can be diffed and restored.
5. Human in control: users can annotate, edit, pin, replace and import graph data, and those edits survive re-parsing.
6. Fully local and private by default.
7. Works in any MCP client (Claude Desktop, Claude Code, Cursor, Windsurf, VS Code) and ships a VS Code extension for the visual side.

**Non-goals (v1)**

- Not a code search engine for huge monorepos (Sourcegraph territory).
- Not a replacement for reading code when an agent actually needs to edit it. Graf points to the exact code, it does not hide it.
- No cloud sync or team collaboration in v1.
- No full type inference or compiler-grade accuracy. Good-enough static structure is the target.

## 4. Users and Use Cases

| User | Need |
|---|---|
| Solo developer using an AI agent daily | Stop paying tokens for the same orientation every session |
| Student or hackathon team | Fast onboarding of an agent into an unfamiliar or inherited codebase |
| Maintainer of a mid-size repo | See impact before changing something ('what breaks if I rename this?') |
| Agent power user | Shape what the agent knows: pin key files, add notes, hide noise |

Top use cases:

1. Agent starts a session and immediately knows the project shape without reading a single file.

2. Agent asks 'where is login handled and what depends on it?' and gets a precise answer in about 500 tokens.
3. Agent asks 'what does changing this function affect?' (impact analysis).
4. Human adds a note: 'payments module is frozen, do not touch', and every future session sees it.
5. Human rolls the graph back to last week's state to see how the architecture changed.

# 5. Core Concept: The Graph Model

## 5.1 Nodes

Every node has a **stable ID**: `kind:path#qualified.name` (example: `function:src/auth/login.js#handleLogin`). Stable IDs let notes, pins and history survive edits, and let rename tracking work.

| Kind | Examples | Stored fields |
|---|---|---|
| project | the repo itself | name, stack, entry points, run commands |
| module/file | `src/auth/login.js` | path, language, size, content hash, one-line summary |
| class | `UserService` | name, extends, implements, line range, summary |
| function/method | `handleLogin` | signature, params, return hint, line range, exported or not, summary |
| route/endpoint | `POST /api/login` | method, path, handler link |
| data model | `User` schema or table | fields, relations |
| config/env | `.env` key names (never values) | key name, where used |
| note (human) | decision, warning, todo | text, author, created, linked nodes |

### 5.2 Edges

`imports`, `exports`, `calls`, `extends`, `implements`, `defines` (file to symbol), `handles` (route to function), `reads/writes` (function to model), `used_by` (derived reverse), `documents` (note to node), `custom` (user-defined).

Every edge carries a **source** flag: `derived` (parser made it) or `human` (user made it), plus a confidence value for heuristic edges such as dynamic calls.

### 5.3 Two layers (the key design decision)

- **Derived layer:** fully regenerable from source. Safe to delete and rebuild at any time.
- **Human layer:** notes, pins, summary overrides, custom edges, ignore rules, locks. Stored separately and re-applied after every re-parse, so user work is never overwritten by the parser.

## 6. System Architecture

```
  Source files  -->  Watcher  -->  Parser  -->  Graph store (SQLite)  <--
MCP server  <--  Agents
                                                ^        |
                                                |        +-->
Snapshot/history engine
                                  VS Code extension / CLI / Web UI (edit,
view, diff, restore)
```

**Components**

1. **Core daemon (Node.js).** Owns the watcher, parser, store and history. One daemon per project.
2. **Parser.** Uses tree-sitter (web-tree-sitter WASM, so no native compilation) for JavaScript, TypeScript, Python, Java and more. Extracts symbols and edges. Languages are plug-in grammar files.
3. **Graph store.** SQLite via Node's built-in `node:sqlite` (Node 22.5+), so zero native dependencies. Fallback: a JSON-lines store for older Node. Lives in `.graf/graph.db`.
4. **MCP server.** Speaks stdio (for Claude Desktop, Claude Code, Cursor) and optionally HTTP. Thin layer over the store.
5. **VS Code extension.** Visual graph, tree view, editing, history, status bar. Talks to the daemon over a local socket.

6. **CLI (`graf`).** init, serve, status, query, export, import, snapshot, restore, diff.

**Tech constraints (matching your stack preference):** vanilla JavaScript, ES modules, JSDoc for types, no bundler, no build step. Only runtime dependencies: web-tree-sitter and grammar WASM files, plus the MCP SDK (or a small hand-written JSON-RPC stdio implementation to avoid even that).

**Suggested folder layout**

```
graf/
  bin/graf.js            CLI entry
  src/
    daemon.js             orchestrates everything
    watcher.js            file watching + debounce
    parser/
      index.js            language router
      extract-js.js       JS/TS symbol and edge extraction
      extract-py.js       Python extraction
    store/
      db.js               SQLite wrapper, migrations
      queries.js          graph queries (neighbors, impact, search)
    history/
      snapshots.js        snapshot, diff, restore
    human/
      overlay.js          notes, pins, overrides re-applied after parse
    mcp/
      server.js           tools + resources
    summarize.js          deterministic + optional LLM summaries
  grammars/               tree-sitter .wasm files
  extension/              VS Code extension (plain JS, no build)
  .graf/                 per-project data (created at runtime)
```

# 7. How Agents Use It (MCP Surface)

## 7.1 The always-in-context strategy

This is the heart of the product, so it needs honesty about how MCP clients behave. Not every client auto-injects MCP *resources* into context. Graf therefore uses three layers so the overview is reliably present:

1. **MCP server `instructions` field.** Sent at connection time. Contains a 300 to 600 token digest plus the rule 'call `overview()` or query Graf before reading files'. Most clients put this straight into the system context.
2. **MCP resource `graf://overview`.** The 2 to 4k token project digest, for clients that attach resources.
3. **Generated pointer file.** Graf auto-writes a short block into `CLAUDE.md`, `AGENTS.md` or `.cursorrules` (opt-in) saying 'this project has an Graf graph, query it first' and embedding the compact digest. This works even with clients that ignore the first two.

All three are regenerated whenever the graph changes meaningfully.

## 7.2 The digest (what stays in context)

Tiered so it never grows beyond a budget:

- **Tier 0 (about 300 tokens):** project name, stack, entry points, run and test commands, top-level folders with one-line purposes, pinned notes.
- **Tier 1 (about 2 to 4k tokens):** module list with summaries, key classes and their roles, route list, main data models, hotspots (most-connected nodes).
- **Tier 2 (on demand):** everything else via tools.

The digest is ranked by graph centrality (how connected a node is) plus recency of change plus user pins, then truncated to the token budget. Budget is configurable.

## 7.3 Tools

| Tool | Purpose | Typical output |
|---|---|---|
| `overview(depth?)` | Get Tier 0 or Tier 1 digest | 300 to 4000 tokens |
| `find_symbol(query, kind?)` | Fuzzy search by name or summary | 5 to 10 lines |
| `get_node(id)` | Full record plus its edges, no source | under 300 tokens |
| `read_node(id, context_lines?)` | Exact source of just that function or class | the code only |
| `neighbors(id, edge_types?, depth?)` | Local subgraph, e.g. callers and callees | small list |

| Tool | Purpose | Typical output |
|---|---|---|
| `impact_of(id)` | Everything transitively depending on a node | grouped list |
| `path_between(a, b)` | How two things connect | short chain |
| `changes_since(ref)` | What changed since a snapshot, time or commit | diff summary |
| `add_note(text, links)` | Agent or human records a decision | confirmation |
| `search_notes(query)` | Retrieve past decisions | matching notes |
| `graph_status()` | Freshness, last update, pending re-index | one line |

Design rule: **every tool response is bounded** (hard cap on items, with 'N more, narrow your query' hints) so a single call can never blow up the context.

# 8. Keeping the Graph Fresh (Real-time Updates)

## 8.1 Update pipeline

1. **Detect.** File watcher (recursive `fs.watch`, with a polling fallback) honouring `.gitignore` and `.grafignore`.
2. **Debounce.** Collect events for about 300 ms so a save-all or a git checkout becomes one batch.
3. **Hash check.** Compare the file's content hash to the stored hash. If unchanged, stop. This kills most false events.
4. **Re-parse only that file.** Delete its old derived nodes and edges, insert the new ones, in a single transaction.
5. **Re-link.** Recompute edges that cross files only for symbols touching the changed file (imports and call targets resolve through a symbol index).
6. **Re-apply the human layer.** Re-attach notes, pins and overrides by stable ID. If a node vanished, move its notes to an 'orphaned' list instead of deleting them.
7. **Notify.** Mark digest dirty, regenerate it (debounced), tell connected extension and MCP clients ( `resources/updated` ).

### 8.2 Modes

| Mode | When | How |
|---|---|---|
| Live (default) | daemon running | watcher, about 100 to 500 ms to fresh |
| On-demand | no daemon | `graph_status()` does a quick stat and hash sweep and updates stale files before answering |
| Git-triggered | CI or commit time | post-commit and post-checkout hooks run `graf sync` |
| Full rebuild | corruption, big refactor, upgrade | `graf rebuild`, derived layer only, human layer untouched |

On daemon start, Graf always runs a **reconcile sweep** (compare file mtimes and hashes against the store) so changes made while it was off are caught.

### 8.3 Handling hard cases

- **Renames and moves:** match by content hash and symbol signature similarity; carry the node ID and its human layer to the new path.
- **Branch switches:** detect the git HEAD change, run the batch pipeline, and optionally keep one graph per branch (config flag).
- **Generated and vendored folders:** ignored by default (`node_modules`, `dist`, `build`, `.git`, lockfiles).
- **Parse errors mid-typing:** keep the last good version of that file's subgraph and mark it stale instead of deleting it.
- **Dynamic code:** edges that are guesses are marked low-confidence and shown that way.

## 9. History and Versioning (Keeping Previous Copies)

### 9.1 What is stored

Two complementary mechanisms:

1. **Change log (event sourcing).** Every node or edge add, update or delete is appended to a `changes` table with a timestamp, the cause (file save, git commit, human edit) and old and new values. This gives fine-grained history and powers `changes_since`.
2. **Snapshots.** Compressed full copies of the graph at meaningful moments:
   - automatically on every git commit (tagged with the commit hash)
   - on a schedule (default hourly if changes happened)
   - before any risky operation (rebuild, import, bulk edit, restore)
   - manually via `graf snapshot 'before refactor'`

## 9.2 Operations

- **List** snapshots with date, label, commit, node and edge counts.
- **Diff** any two snapshots (or a snapshot and now): added, removed and modified nodes and edges, grouped by module.
- **Restore** a snapshot fully, or just selected parts (only the human layer, only one module). Restoring always takes a safety snapshot first, so a restore is itself undoable.
- **Time-travel read:** MCP tools accept an optional `at` parameter, so an agent can ask 'how did the auth module look last week?'.

## 9.3 Retention

Default policy: keep all snapshots for 7 days, then daily for 30 days, then weekly for a year, and never prune labelled or commit-tagged snapshots unless the user asks. Storage is small because snapshots are compressed and large projects are still only megabytes of graph data. Retention is configurable.

## 9.4 Optional git-friendly export

`graf export` writes the **human layer** as plain, sorted JSON (`.graf/human.json`) that can be committed to git so teams share notes and pins. The derived layer stays out of git because it is rebuildable.

# 10. Editing, Replacing and Working with the Graph

## 10.1 What users can do

| Action | Details |
|---|---|
| Annotate | Attach a note to any node: decision, warning, todo, context |
| Override summary | Replace the auto-generated one-liner with your own; marked `locked` so the parser and LLM never overwrite it |
| Pin | Force a node into the always-in-context digest |
| Hide or ignore | Remove noisy nodes or whole folders from the digest and queries |
| Add custom edges | Link things the parser cannot see (e.g. 'frontend form submits to this route', or 'cron job triggers this') |
| Rename or merge nodes | Fix mis-detected symbols or merge duplicates; stable ID aliasing keeps history intact |
| Define groups | Name architecture layers or domains ('Billing', 'Auth') and assign nodes to them |
| Replace a subgraph | Import a JSON file to replace one module or the whole human layer |
| Bulk edit | Edit exported JSON in any editor, then `graf import` it |

## 10.2 Conflict rules

1. **Human beats derived.** A locked human value is never overwritten.
2. **Unlocked summaries** are regenerated only when the underlying code hash changes.
3. If the code under a human-edited node changes a lot, Graf flags it **'possibly stale'** instead of silently trusting or discarding the edit.
4. Every import shows a diff preview and requires confirmation, and takes a safety snapshot first.

### 10.3 Interfaces for editing

1. **VS Code extension** (primary; see section 11).
2. **CLI:** `graf note add`, `graf pin`, `graf edge add`, `graf export`, `graf import`.
3. **Plain JSON files** in `.graf/` for power users.
4. **Through the agent:** `add_note` and (permission-gated) `edit_node` MCP tools so the agent can record decisions as it works. Agent-written entries are tagged `author: agent` so humans can review and approve them.

## 11. VS Code Extension

### 11.1 Features

- **Graph view (webview):** interactive node-link diagram using a vendored Cytoscape.js or D3 file (no build step). Filters by kind, folder, edge type and recency. Click a node to open the source. Colour by module, size by connectivity, dashed lines for low-confidence edges.
- **Explorer tree view:** Graf panel in the sidebar: modules, classes, functions, notes, pins, orphans, with right-click actions (add note, pin, hide, show impact, show callers).
- **Live status bar item:** 'Graf: fresh | 1,204 nodes | updated 2s ago' with click-to-open and a spinner during re-index.
- **Node inspector:** summary, edges, notes, history of that node, edit buttons.
- **History panel:** snapshot timeline, diff viewer (colour-coded added, removed, changed), one-click restore with confirmation.
- **Impact preview:** right-click a symbol, then 'Show impact', to highlight everything affected.
- **Agent activity lens (stretch):** show which nodes the agent queried in the last session.
- **One-click MCP setup:** command 'Graf: Connect to Claude / Cursor' writes the right MCP config entry for the chosen client.
- **Token savings meter:** estimated tokens saved this session (see section 12).

### 11.2 How it is built

- Plain JavaScript extension (`extension.js`, `package.json` contributes), no bundler. Packaged with `vsce` for the Marketplace and Open VSX (so it also works in Cursor, Windsurf and VSCodium).

- Talks to the project's daemon over a local Unix socket or named pipe (or localhost HTTP with a random token). If no daemon is running, the extension starts one.
- Webview uses message passing with the extension host; graph data is sent as compact JSON, and large graphs are windowed (show a focused neighbourhood, not 10,000 nodes at once).
- Commands: `Graf: Init`, `Graf: Rebuild`, `Graf: Snapshot`, `Graf: Open Graph`, `Graf: Diff Snapshots`, `Graf: Restore`, `Graf: Export`, `Graf: Import`.

## 12. Efficiency: What to Expect and How to Prove It

These are planning estimates, not measured results. Part of the MVP is a benchmark harness that produces real numbers.

### 12.1 Token efficiency (estimates)

Assume a 500-file project of about 150,000 tokens of source.

| Scenario | Without Graf | With Graf |
|---|---|---|
| Session orientation | 20k to 60k tokens of file reads | 0.3k to 4k (digest) |
| 'Where is X and who uses it?' | 5k to 15k (grep plus several file reads) | 0.5k to 1.5k ( `find_symbol` plus `neighbors` ) |
| 'What breaks if I change Y?' | 10k to 40k | about 1k ( `impact_of` ) |
| Re-verifying structure mid-session | repeated reads | free, already in context |

Realistic overall target: **80 to 95 percent fewer orientation tokens**, with the largest gains on medium and large projects. Tiny projects (under about 20 files) gain little. The agent still reads real source when editing, via `read_node`, which returns only the relevant function.

### 12.2 Performance targets

| Metric | Target |
|---|---|
| Initial index, 1,000 files | under 15 seconds |
| Initial index, 10,000 files | under 2 minutes |
| Incremental update after a save | under 300 ms to consistent |
| Query latency (any tool) | under 50 ms |
| Daemon memory | under 150 MB for typical projects |
| Graph DB size | roughly 1 to 3 MB per 1,000 files, plus snapshots |

### 12.3 Benchmark harness

A scripted set of 20 to 30 realistic tasks per sample repo ('add a field to the User model', 'find and fix the login bug'), run with and without Graf, measuring tokens used, tool calls, wall time and task success. Publish the results. This is also your best marketing material.

### 12.4 Summary generation strategy

- **Default, free, deterministic:** summary is built from the signature, docstring or leading comment, and name heuristics. No LLM calls.
- **Optional LLM summaries:** user supplies an API key or points to a local model; summaries are generated lazily only for nodes that matter (high centrality or queried), cached by content hash, and never regenerated unless the code changes.
- Result: works offline and free on day one, and gets smarter if the user opts in.

## 13. Data Model (SQLite Sketch)

```
CREATE TABLE nodes (
  id TEXT PRIMARY KEY,          -- kind:path#qualified.name
  kind TEXT NOT NULL,
  path TEXT,
  name TEXT NOT NULL,
  signature TEXT,
  summary TEXT,
  summary_locked INTEGER DEFAULT 0,
```

```
  line_start INTEGER, line_end INTEGER,
  content_hash TEXT,
  meta JSON,
  layer TEXT DEFAULT 'derived', -- derived | human
  stale INTEGER DEFAULT 0,
  updated_at INTEGER
);
CREATE TABLE edges (
  src TEXT, dst TEXT, type TEXT,
  source TEXT DEFAULT 'derived', -- derived | human
  confidence REAL DEFAULT 1.0,
  PRIMARY KEY (src, dst, type)
);
CREATE TABLE files (
  path TEXT PRIMARY KEY, hash TEXT, mtime INTEGER, lang TEXT, parse_ok
INTEGER
);
CREATE TABLE notes (
  id INTEGER PRIMARY KEY, node_id TEXT, text TEXT,
  author TEXT,            -- human | agent
  approved INTEGER DEFAULT 1, created_at INTEGER
);
CREATE TABLE pins (node_id TEXT PRIMARY KEY, weight INTEGER);
CREATE TABLE changes (
  seq INTEGER PRIMARY KEY AUTOINCREMENT, ts INTEGER,
  cause TEXT, entity TEXT, entity_id TEXT, old JSON, new JSON
);
CREATE TABLE snapshots (
  id INTEGER PRIMARY KEY, ts INTEGER, label TEXT,
  git_commit TEXT, node_count INTEGER, edge_count INTEGER, blob BLOB
);
CREATE INDEX idx_edges_dst ON edges(dst);
CREATE INDEX idx_nodes_name ON nodes(name);
```

Full-text search on names and summaries uses SQLite FTS5.

## 14. Security and Privacy

- 100 percent local by default. No telemetry, no network calls unless the user enables LLM summaries.
- Respects `.gitignore`. Never indexes `.env`, key files or certificates; for config files stores **key names only, never values**.

- A secret-pattern scan blocks anything that looks like a token from entering summaries or notes.
- MCP HTTP mode (if enabled) binds to localhost only with a random token.
- Agent write tools (`add_note`, `edit_node`) are scoped and logged. Edits by agents are tagged and reversible via history.
- Treat note and summary text as untrusted data in prompts. Graf content returned to agents is clearly delimited, to reduce prompt-injection risk from code comments.

## 15. Milestones

| Phase | Scope | Rough time (solo) |
|---|---|---|
| 0. Spike | Parse JS with tree-sitter, store in SQLite, expose `overview` and `find_symbol` over stdio MCP; test with Claude Desktop or Cursor | 3 to 4 days |
| 1. Core MVP | Full node and edge model, watcher with hash-based incremental updates, all read tools, digest tiers, `.gitignore` handling, CLI | 1.5 to 2 weeks |
| 2. History and human layer | Change log, snapshots, diff, restore, notes, pins, summary locks, export and import | 1 to 1.5 weeks |
| 3. VS Code extension | Tree view, graph webview, status bar, history panel, one-click MCP setup | 1.5 to 2 weeks |
| 4. Polish and benchmark | TS and Python support, benchmark harness, docs, demo video, Marketplace release | 1 week |
| 5. Later | Optional LLM summaries, per-branch graphs, team sync of the human layer, more languages | ongoing |

**MVP definition of done:** install, run `graf init`, connect to an MCP client, and an agent answers 'where is X and what uses it?' in under 1.5k tokens without reading a file, while the graph updates live as you edit.

## 16. Success Metrics

- Orientation tokens reduced by at least 80 percent in the benchmark suite.
- Task success rate with Graf is equal to or better than without it.
- Graph freshness: 99 percent of saves reflected in under 1 second.
- Time to first value: under 2 minutes from install to a working agent query.
- Adoption: weekly active projects, notes created per project (a proxy for the human layer's value), restores performed.

## 17. Risks and Mitigations

| Risk | Mitigation |
|---|---|
| Clients do not auto-load resources, so the overview is not 'always in context' | Three-layer strategy in 7.1 (instructions field, resource, generated pointer file) |
| Crowded space (repo maps, code-graph MCPs) | Differentiate on zero-config, live updates, history, human-editable layer and decision notes |
| Parser inaccuracy on dynamic languages | Confidence scores, mark guesses, let humans add and fix edges |
| Stale graph misleads the agent | `graph_status()` freshness, stale flags, on-demand sweep, reconcile on startup |
| Graph grows too large to digest | Centrality ranking, hard token budgets, bounded tool outputs |
| Agents write bad notes | Agent notes tagged and reviewable, with an approve or reject flow |
| Scope creep | Strict phase gates; extension only after core and history are solid |

## 18. Assumptions Made

- Primary stack is Node.js 22 or newer in vanilla JavaScript with no build tools (matches your usual preference).
- First supported languages are JavaScript and TypeScript, then Python.

- Local-first, single-developer scope for v1.
- Free, deterministic summaries by default, with LLM summaries as an opt-in extra.
- The name Graf was chosen by the author.

## 19. Open Questions

1. Should v1 also support Java and C#, since you are a BCA student and many college projects use them?
2. Do you want a standalone web UI in addition to the VS Code extension (it could be hosted on Dilect)?
3. Should the human layer be committed to git by default for team sharing?
4. Open source from day one (best for adoption and a portfolio piece) or closed during early development?

## 20. v0.2 Merge: What Changed

This version merges the strongest ideas from the ChatGPT-written Graf PRD v1.0 into our design. Where a rule in this section conflicts with sections 1 to 19, this section wins.

### 20.1 Provenance on every fact

Every node and edge carries a `provenance` label:

- **verified:** confirmed by source code or language tooling
- **inferred:** produced by static analysis or a heuristic (keep the numeric confidence here)
- **ai_generated:** proposed by a model and not yet verified
- **manual:** added or approved by a human

Rule: Graf never presents an inferred or AI-generated fact as verified. Every tool result returns the label, and the digest marks low-trust items. This replaces the simple derived/human flag in section 5.

### 20.2 Approval rules for AI and agent edits

| Operation | Default behavior |
| --- | --- |
| Read or query the graph | Automatic |

| Operation | Default behavior |
|---|---|
| Add verified source facts | Automatic |
| Create an AI summary | Automatic, labelled ai_generated |
| Add an agent note | Automatic, tagged author agent, reviewable |
| Edit human notes or locked values | Approval required |
| Delete or replace nodes | Approval required |
| Restore a version | Approval required |
| Bulk import | Approval required, with diff preview |
| Modify source code | Never done by Graf |

Agents use `propose_changes`. Each proposal contains the change set, a short rationale, evidence, confidence and expected impact. A human applies it from the extension or CLI.

## 20.3 Branch and commit awareness (now core)

Every graph version records the git branch and commit (or detached HEAD). A branch switch loads that branch's graph state or computes the difference, and entities from different branches never mix. This was an optional flag in v0.1 and is now always on.

## 20.4 Versions and restore semantics

- Each version references its parent version, forming a chain.
- Restoring a whole version, one node, one edge or one memory record creates a **new version** and never erases the history it restored from.
- Deleted entities use a `deleted_at` tombstone, not a hard delete.
- No snapshot per keystroke. Snapshots happen at git commits, on a schedule, manually, and before risky operations.
- Schema additions: `graph_versions(id, version_number, parent_version_id, git_branch, git_commit, summary, created_at)`; the `changes` table gains `version_id`, `before`, `after` and `source`.

## 20.5 Memory as a first-class layer

- **Structural memory:** automatically extracted facts, refreshed from code.

- **Semantic memory:** decisions, conventions, rejected options, known limits and pending decisions that code cannot show.
- A memory record has a title, content, source (developer, agent or imported), status (active or archived), linked nodes, supporting evidence and timestamps.
- Lifecycle: create, read, update, archive, restore, delete, link to nodes, inspect history.
- A manually approved memory is never overwritten by an automatic summary.
- Retrieval priority: task relevance, explicit developer instructions, verified code facts, recent changes, architecture decisions, history, then low-confidence inferences.

## 20.6 Stable symbol identity

Line numbers are never identity. Identity comes from repository, language, kind, qualified name, parent symbol and file identity, with the content hash stored separately. For renames and moves, use a structural fingerprint. Ambiguous matches are shown to the user as candidates and never silently merged.

## 20.7 Token-budgeted retrieval and four context levels

| Level | Content |
|---|---|
| 1. Overview | Short project summary |
| 2. Module context | Relevant architecture and dependencies |
| 3. Symbol context | Specific classes, functions and their relationships |
| 4. Source context | Exact snippets with locations |

Agents request the lowest level that answers the question. The `get_context` tool accepts `query`, `max_tokens`, `include_dependencies`, `include_memory` and `include_source_locations`. Its flow: identify intent, search nodes, traverse relationships, fetch linked memories, attach evidence, rank, truncate to the budget, and return structured results. The whole graph is never returned by default.

## 20.8 MCP tool surface, capped at 12

Every tool description costs context, so v0.2 caps the agent-facing tools at twelve: `overview`, `search`, `get_node`, `neighbors`, `impact_of`, `read_node`, `get_context`, `memory_search`, `memory_write`, `propose_changes`, `changes_since`, `sync_status`. This

supersedes the longer list in section 7.3. Applying changes and restoring versions are human-only actions in the extension and CLI, not agent tools.

### 20.9 Extra safeguards

- Validate every tool argument, restrict file access to the repository root, and block path traversal.
- Log every graph mutation.
- Treat comments, docs and imported content as untrusted data that can never override rules or permissions.
- If a file fails to parse, keep its last good state, mark it stale and show the error. Never replace valid data with an empty graph.
- Export bundle contains graph, memory, history, schema version and repository identity, and no source files unless asked. Formats: JSON, JSONL and a Markdown architecture report (GraphML later).

### 20.10 Success measure

A token reduction only counts if task correctness holds. The benchmark records files read per task, tool calls, discovery tokens, query latency, task success and the rate of stale information.

### 20.11 Deliberately not adopted from the other PRD

- LSP integration: after the MVP.
- Electron or Tauri desktop app and web dashboard: after the MVP (a dashboard could be hosted on Dilect).
- TypeScript, React and Zod toolchain: we stay vanilla JavaScript with JSDoc types and no build step.
- The 12-week all-in roadmap: we keep the phased plan in section 15.
- Pricing and business model: premature before real users.
- Vector search: optional, later.
- Viewer, Editor and Administrator roles: in v1 the local user is the administrator.

### 20.12 First proof of concept (3 to 4 days)

1. Index a real JavaScript project into SQLite.
2. Change one file and confirm only the affected entities update.

3. Create a version and show the diff against the previous one.
4. Ask an agent through MCP to explain a module without it reading the repository.

Build the core engine first. The MCP server and the VS Code extension are interfaces to it.